CAPSTONE PROJECT — APRIL 2026

ReliefOps

Comprehensive Architecture & Software Engineering Report

A full-stack disaster response platform connecting citizens, volunteers, and admins — with real-time resource allocation, intelligent matching strategies, and a live operations dashboard.

Project Team

Pushpendra S. Parihar · Mouli Srivastava · Gauri Jindal · Meghna Nair · Yashvi Goyal

Lead: Mouli Srivastava

GitHub Repository github.com/MouliSrivastava05/ReliefOps

Live Demo relief-ops-two.vercel.app

CONTENTS —

01 Introduction & Problem Statement

02 Architectural Design Patterns

2.1 Model-View-Controller (MVC)

2.2 Singleton Pattern

2.3 Factory Method Pattern

2.4 Strategy Pattern (Polymorphism)

2.5 State Pattern

2.6 Observer Pattern

03 Implementation of SOLID Principles

04 System Overview & Technical Diagrams

Fig. 1 Use Case Diagram

Fig. 2 Entity-Relationship Diagram

Fig. 3 UML Class Diagram

Fig. 4 Sequence Diagram

05 Tech Stack & API Surface

01 — OVERVIEW — Introduction & Problem Statement

ReliefOps addresses a critical bottleneck in Disaster Response: the absence of a unified, real-time coordination layer between citizens in need, volunteer responders, and administrative command. During active crises, volunteers are unsure where to deploy, shelters are unaware of incoming capacity demands, and high-priority cases are delayed due to duplicate or misrouted requests.

ReliefOps acts as a live operational hub by intelligently routing crisis requests, matching resources to needs, and surfacing live status across all stakeholders.

Problem	Challenge It Solves
No volunteer coordination	Volunteer portal with RBAC, interest-raising, and approval workflows
Shelter capacity blindness	Real-time resource tracking with atomic allocation guards
Resource misallocation	Pluggable allocation strategies — Greedy vs. Severity-first
Duplicate high-priority cases	Request fingerprinting by (citizenId, type, locationKey)
No live situational view	Admin dashboard with Google Maps integration and live event feed

The system is built as a full-stack **Next.js 15** application with **TypeScript 5**, MongoDB via Mongoose, NextAuth.js v4 for JWT-based Role-Based Access Control (RBAC), and TanStack React Query for server-state synchronisation. All core domain logic is implemented following strict OOP and SOLID principles with five canonical design patterns.

02 — ARCHITECTURE — Architectural Design Patterns

ReliefOps enforces six proven design patterns across its domain layer, making the codebase robust, testable, and open for extension without modification.

STRUCTURAL

Model-View-Controller (MVC)

The full-stack application runs on Next.js 15's App Router, separating concerns cleanly. The **Model** (Mongoose schemas + repositories in `src/domain/models`) encapsulates all data logic. The **View** (React 19 + Tailwind components in `src/components`) renders live dashboards, maps, and portals. The **Controller** (Next.js API route handlers in `src/app/api`) validates requests, delegates to services, and returns structured responses.

CREATIONAL

Singleton Pattern

Implemented in `src/lib/mongodb.ts` via `connectMongo()`. A single Mongoose connection is established once and reused across all serverless function invocations — preventing connection exhaustion under concurrent load. NextAuth session state and Redux Toolkit store also behave as global singletons on the client.

CREATIONAL

Factory Method Pattern

Implemented in `src/domain/patterns/factory/RequestFactory`. The factory resolves crisis type strings (`'medical'`, `'shelter'`, `'food'`) to their concrete subclass constructors — MedicalRequest, ShelterRequest, FoodRequest — without hard-coding instantiation logic anywhere in the service layer. Covered by `RequestFactory.test.ts`.

BEHAVIORAL

Strategy Pattern (Polymorphism)

Defined in `src/domain/patterns/strategy/`. The abstract **AllocationStrategy** interface exposes a single `allocate(requests, resources)` method. **GreedyStrategy** maximises throughput by matching the first available resource. **SeverityStrategy** reorders by severity weight before matching. A new DistanceStrategy requires only implementing the interface — zero changes to AllocationService.

BEHAVIORAL

State Pattern

Implemented in `src/domain/patterns/state/RequestStateMachine` and RequestStates. Each crisis request moves through legal transitions only: **PENDING** → **QUEUED** → **ALLOCATED** → **CLOSED**. Illegal transitions (e.g. CLOSED → QUEUED) throw a `StateTransitionError`, enforced and tested in `RequestStateMachine.test.ts`.

BEHAVIORAL

Observer Pattern

Implemented in `src/domain/patterns/observer/ObserverManager` and DashboardObserver. When the allocation engine completes, it notifies all registered observers — the dashboard observer pushes allocation events to the live feed without any coupling between AllocationService and the UI layer.

03 — ENGINEERING — Implementation of SOLID Principles

ReliefOps strictly adheres to the SOLID software design philosophy across every layer, ensuring each component is clean, maintainable, and future-proof.

S **Single Responsibility Principle (SRP)**
Responsibilities are strictly separated. API route handlers in `src/app/api` handle HTTP only. AllocationService, RequestService, VolunteerService orchestrate use cases. RequestRepository, ResourceRepository, UserRepository manage persistence. MedicalDataProcessor is never mixed with AnalysisService.

O **Open/Closed Principle (OCP)**
The Strategy pattern makes AllocationService open for extension: adding DistanceStrategy requires only implementing AllocationStrategy — the allocation pipeline never needs editing. Similarly, `RequestFactory.register()` adds new request types without modifying `create()`.

L **Liskov Substitution Principle (LSP)**
Any pipeline expecting a CrisisRequest can receive MedicalRequest, ShelterRequest, or FoodRequest without failure. Each subclass implements `priorityWeight()` and `getResourceType()` correctly, satisfying the parent contract regardless of internal specialisation.

I **Interface Segregation Principle (ISP)**
Separate minimal interfaces — IRequestRepository, IResourceRepository, IUserRepository — ensure no module is forced to depend on methods it does not use. The AllocationStrategy interface exposes only `allocate()`, never storage or auth concerns.

D **Dependency Inversion Principle (DIP)**
AllocationService depends on the abstract AllocationStrategy interface, never on GreedyStrategy or SeverityStrategy directly. Services receive repositories via interface types, not concrete Mongoose model imports — safeguarding the domain if the persistence layer is swapped.

04 — SYSTEM DIAGRAMS — System Overview & Technical Diagrams

Use Case Diagram

Actor interactions — ReliefOps Platform

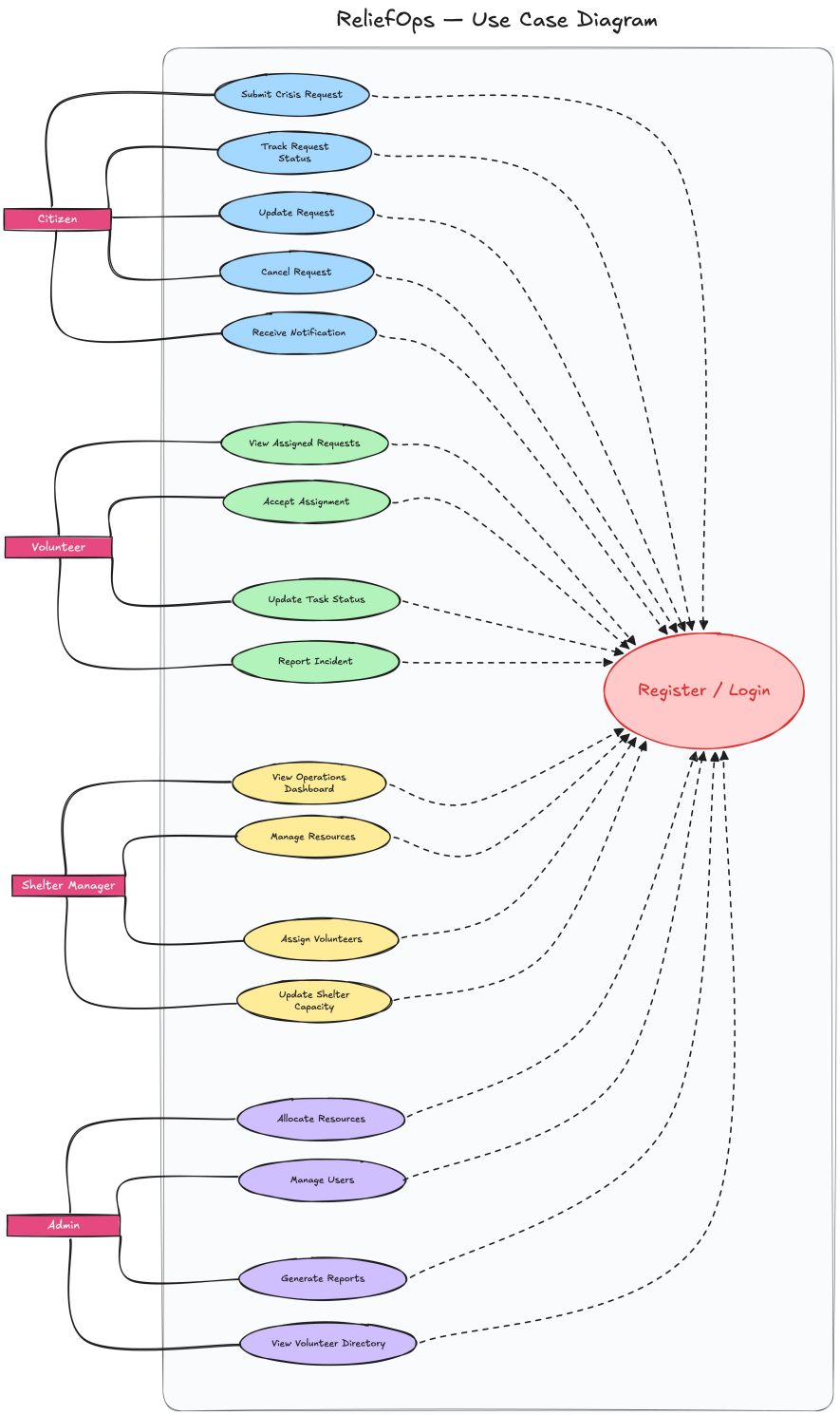

Figure 1 — Illustrates how Citizens, Volunteers, Admins, and Shelter Managers interact with the system. Key flows: submit crisis request, trigger resource allocation, approve volunteer applications, monitor live dashboard, audit allocation events.

- Citizens: submit-request, track-status
- Volunteers: portal, raise interest, profile management
- Admins: dashboard, requests, resources, volunteers (full control)
- Shelter Managers: dashboard, resources (scoped access)

Entity-Relationship (ER) Diagram

MongoDB / Mongoose document schema

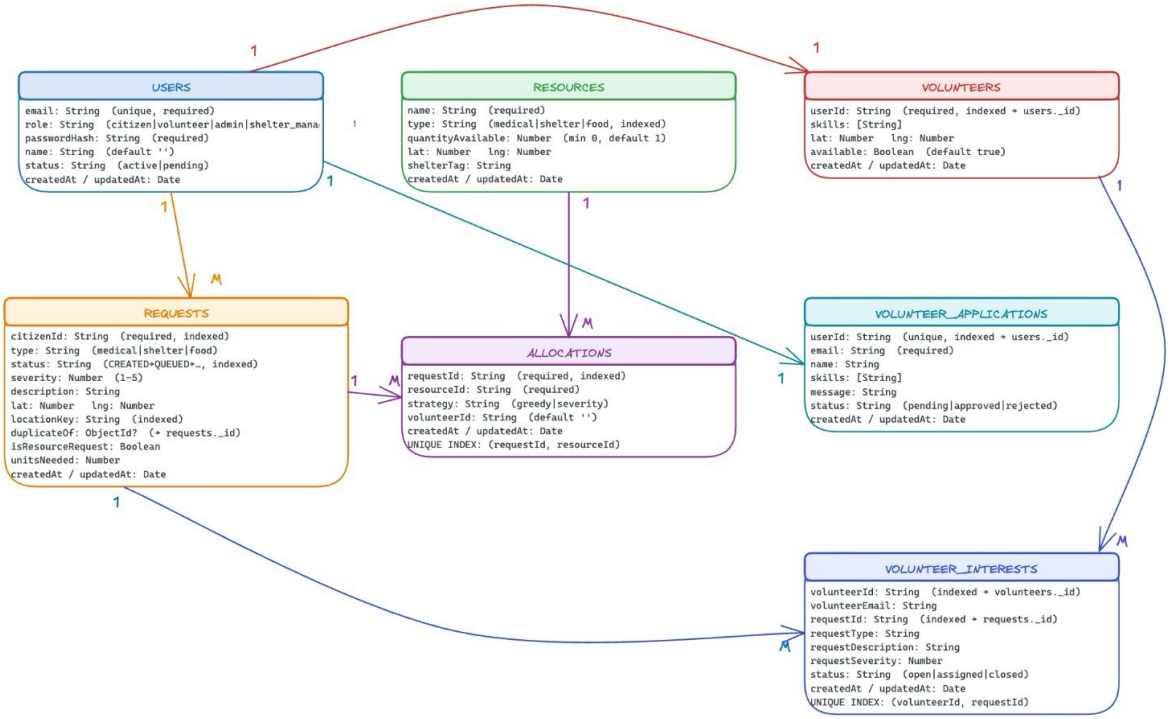

Figure 2 — Users assume Citizen, Volunteer, or Admin roles. Citizens create CrisisRequests (Medical / Shelter / Food) which enter an allocation queue. AllocationService matches requests to Resources, producing AllocationEvents captured by the Observer. Each Resource tracks quantityAvailable with atomic guards.

- User → role: citizen | volunteer | admin | shelter_manager
- CrisisRequest → type: medical | shelter | food | status: PENDING | QUEUED | ALLOCATED | CLOSED
- Resource → category, quantityAvailable (atomic $inc guard)
- AllocationEvent → requestId, resourceId, strategy, timestamp
- VolunteerApplication → volunteerId, status: pending | approved | rejected

UML Class Diagram

Factory, Strategy, State & Observer pattern hierarchy

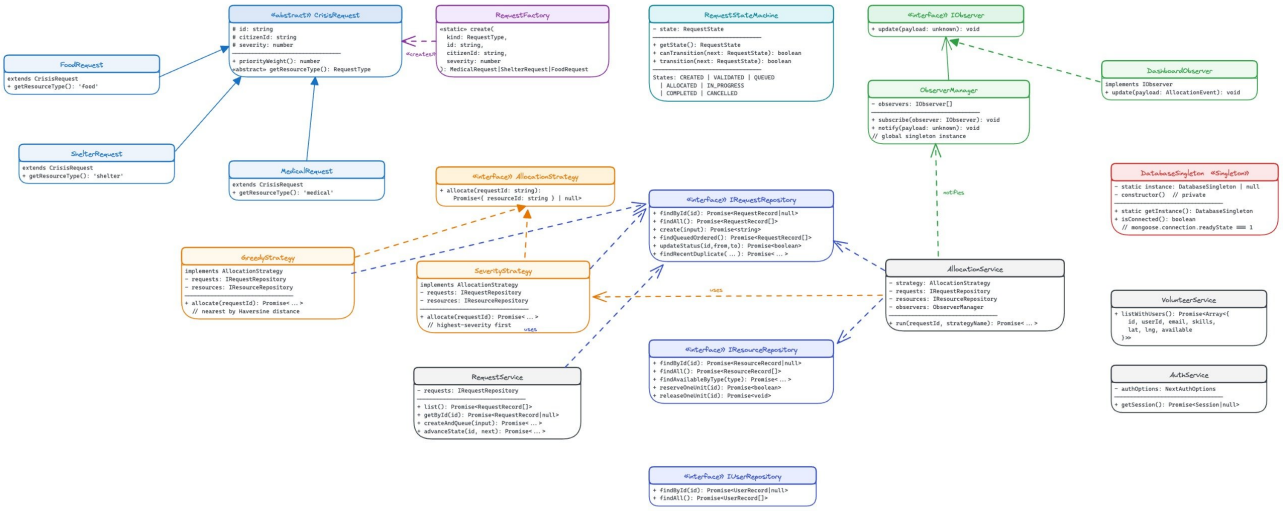

Figure 3 — Abstract CrisisRequest extended by MedicalRequest, ShelterRequest, FoodRequest. RequestFactory creates instances via type registry. AllocationService consumes abstract AllocationStrategy (GreedyStrategy | SeverityStrategy). RequestStateMachine enforces legal transitions. ObserverManager notifies DashboardObserver on allocation events.

- CrisisRequest (Abstract): +priorityWeight(): int, +getResourceType(): string
- MedicalRequest | ShelterRequest | FoodRequest — strategy subclasses
- AllocationStrategy (Interface): +allocate(requests, resources): AllocationResult[]
- RequestStateMachine: +transition(request, newState): void — throws on illegal moves
- ObserverManager: +notify(event): void — DashboardObserver pushes to live feed

Sequence Diagram

RBAC-gated allocation flow for a crisis request session

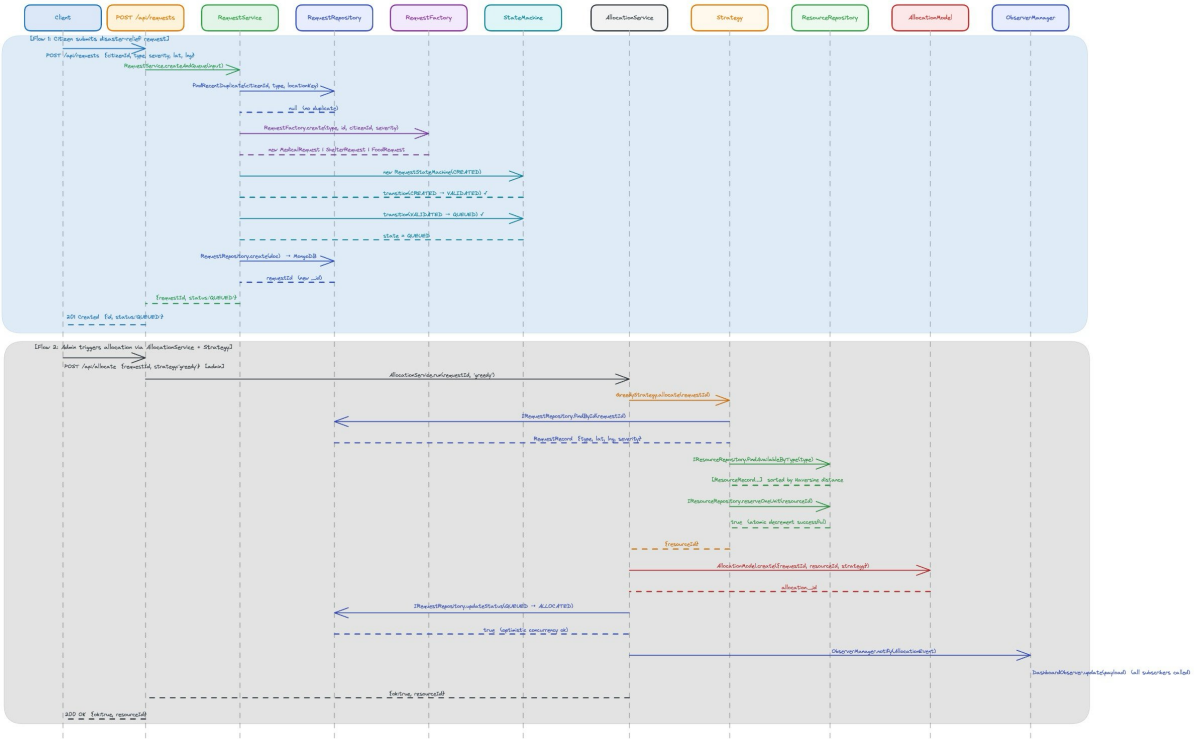

Figure 4 — Full session: Citizen submits POST /api/requests (JWT middleware enforces 'citizen' role), RequestFactory creates typed subclass, RequestStateMachine sets PENDING → QUEUED, Admin triggers POST /api/allocate, AllocationService runs selected Strategy, atomic MongoDB $inc guard executes, ObserverManager notifies DashboardObserver, live event appears on Admin dashboard.

- Middleware: JWT decoded → role checked → 401 on mismatch
- RequestFactory: type string → concrete CrisisRequest subclass
- AllocationService: strategy.allocate() → $inc: { quantityAvailable: -1 } with $gt: 0 guard
- Compensation rollback: releaseOneUnit() called on any downstream failure
- Observer: DashboardObserver.update(event) pushes to live feed without coupling

05 — TECH STACK & API SURFACE — Technology Decisions & Key API Endpoints

Layer	Technology
Language	TypeScript 5
Framework	Next.js 15 — App Router, React 19
UI & Styling	Tailwind CSS 3
Server State	TanStack React Query v5
Client State	Redux Toolkit
Auth	NextAuth.js v4 — Credentials + JWT
Database	MongoDB via Mongoose 8
In-Memory DB	mongodb-memory-server (auto-fallback)
Map	Google Maps via @vis.gl/react-google-maps
Tests	Jest 29 + jest-environment-jsdom

GET/POST	/api/requests	Any role	List / submit crisis requests
GET/POST	/api/resources	Any / Admin+Shelter	List / add resources
POST	/api/allocate	Admin, Shelter Mgr	Run allocation engine
GET	/api/volunteers	Admin	List volunteers with user info
GET/POST	/api/volunteer-applications	Volunteer/Admin	Apply or manage applications
GET/POST	/api/volunteer-interests	Volunteer/Admin	Raise or manage interest
GET/POST	/api/resource-requests	Volunteer	Request a resource
POST	/api/auth/[...nextauth]	Public	NextAuth sign-in/out/session

Made with ❤ for disaster relief coordination · Capstone Project 2026